

Projeto de Circuitos Sequenciais

Prática

Prof. Roberto de Matos

roberto.matos@ifsc.edu.br



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
São José

- Entender as más práticas de projeto e como solucionar.
 - Uso indevido do *reset* assíncrono
 - *Gated clock*
 - *Clock* derivado
- Observação sobre o Consumo de energia
- Diretrizes de síntese



Objetivo

- Entender as más práticas de projeto e como solucionar.
 - Uso indevido do *reset* assíncrono
 - *Gated clock*
 - *Clock* derivado
- Observação sobre o Consumo de energia
- Diretrizes de síntese

Leitura Recomendada:

Capítulo 9 do livro: *RTL Hardware Design Using VHDL*



Uso indevido do *reset* assíncrono

Uso indevido do *reset* assíncrono

- **Má prática:** usar o *reset* para zerar o registro durante a operação normal.
- **Exemplo:** contador *mod-10* problemático:
 - Zerar o registro imediatamente após o contador atingir 1010.



- **Má prática:** usar o *reset* para zerar o registro durante a operação normal.
- **Exemplo:** contador *mod-10* problemático:
 - Zerar o registro imediatamente após o contador atingir 1010.

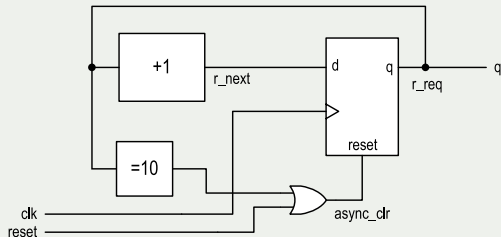
```
1  -----
2  -- Listing 9.1 async clear
3  -----
4  library ieee;
5  use ieee.std_logic_1164.all;
6  use ieee.numeric_std.all;
7  entity mod10_counter is
8      port(
9          clk, reset: in std_logic;
10         q: out std_logic_vector(3 downto 0)
11     );
12 end mod10_counter;
```

```
14 architecture poor_async_arch of mod10_counter is
15     signal r_reg: unsigned(3 downto 0);
16     signal r_next: unsigned(3 downto 0);
17     signal async_clr: std_logic;
18 begin
19     -- register
20     process(clk, async_clr)
21     begin
22         if (async_clr='1') then
23             r_reg <= (others=>'0');
24         elsif (clk'event and clk='1') then
25             r_reg <= r_next;
26         end if;
27     end process;
28     -- asynchronous clear
29     async_clr <= '1' when (reset='1' or r_reg="1010") else
30         '0';
31     -- next state logic
32     r_next <= r_reg + 1;
33     -- output logic
34     q <= std_logic_vector(r_reg);
35 end poor_async_arch;
```



Uso indevido do *reset* assíncrono

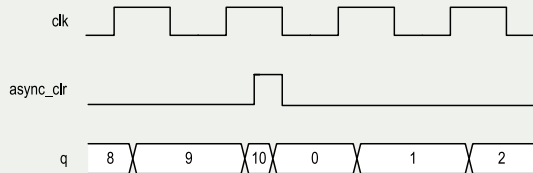
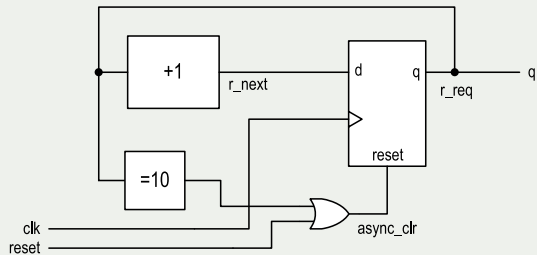
```
1  -----
2  -- Listing 9.1 async clear
3  -----
4  library ieee;
5  use ieee.std_logic_1164.all;
6  use ieee.numeric_std.all;
7  entity mod10_counter is
8      port(
9          clk, reset: in std_logic;
10         q: out std_logic_vector(3 downto 0)
11     );
12 end mod10_counter;
```



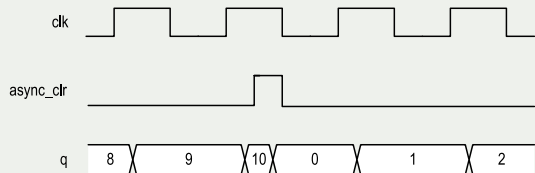
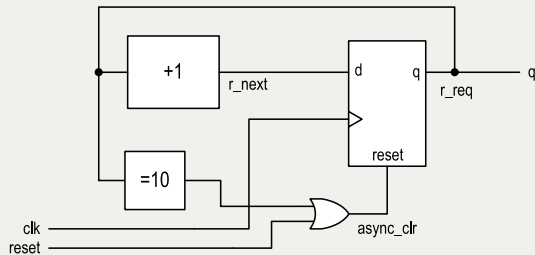
```
14 architecture poor_async_arch of mod10_counter is
15     signal r_reg: unsigned(3 downto 0);
16     signal r_next: unsigned(3 downto 0);
17     signal async_clr: std_logic;
18 begin
19     -- register
20     process(clk, async_clr)
21     begin
22         if (async_clr='1') then
23             r_reg <= (others=>'0');
24         elsif (clk'event and clk='1') then
25             r_reg <= r_next;
26         end if;
27     end process;
28     -- asynchronous clear
29     async_clr <= '1' when (reset='1' or r_reg="1010") else
30         '0';
31     -- next state logic
32     r_next <= r_reg + 1;
33     -- output logic
34     q <= std_logic_vector(r_reg);
35 end poor_async_arch;
```



Uso indevido do *reset* assíncrono



Uso indevido do *reset* assíncrono



■ Problemas:

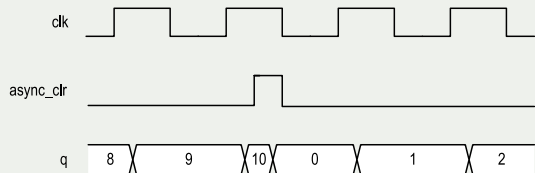
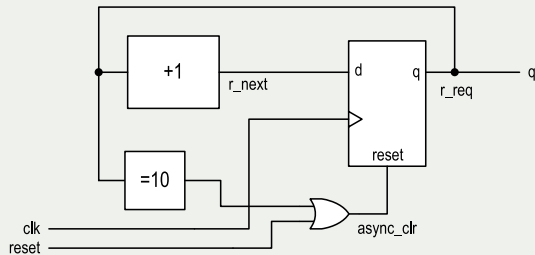
- *Glitches* na transição 1001(9) → 0000(0)
- *Glitches* no sinal `aync_clr` podem zerar o contador
- Como fazer a análise de *timing*? (taxa máxima de *clock*).

■ Regra:

- O Reset assíncrono deve ser usado apenas na inicialização do sistema (*power-on*)



Uso indevido do *reset* assíncrono



■ Problemas:

- *Glitches* na transição 1001(9) → 0000(0)
- *Glitches* no sinal `aync_clr` podem zerar o contador
- Como fazer a análise de *timing*? (taxa máxima de *clock*).

■ Regra:

- O Reset assíncrono deve ser usado apenas na inicialização do sistema (*power-on*)

Solução:

Carregar 0000(0) de forma síncrona.



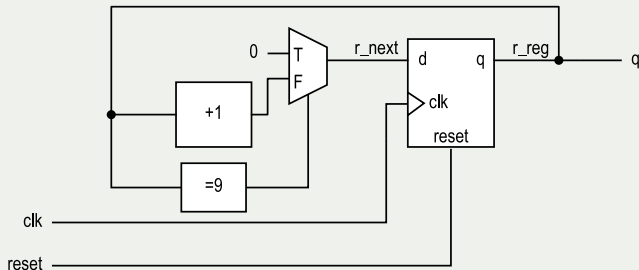
■ Solução:

```
38 -----
39 -- Listing 9.2
40 -----
41 architecture two_seg_arch of mod10_counter is
42     signal r_reg: unsigned(3 downto 0);
43     signal r_next: unsigned(3 downto 0);
44 begin
45     -- register
46     process(clk,reset)
47     begin
48         if (reset='1') then
49             r_reg <= (others=>'0');
50         elsif (clk'event and clk='1') then
51             r_reg <= r_next;
52         end if;
53     end process;
54     -- next-state logic
55     r_next <= (others=>'0') when r_reg=9 else
56         r_reg + 1;
57     -- output logic
58     q <= std_logic_vector(r_reg);
59 end two_seg_arch;
```



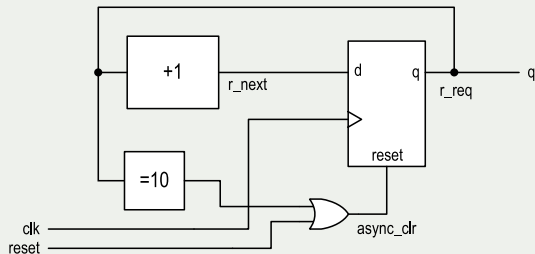
■ Solução:

```
38 -----
39 -- Listing 9.2
40 -----
41 architecture two_seg_arch of mod10_counter is
42     signal r_reg: unsigned(3 downto 0);
43     signal r_next: unsigned(3 downto 0);
44 begin
45     -- register
46     process(clk,reset)
47     begin
48         if (reset='1') then
49             r_reg <= (others=>'0');
50         elsif (clk'event and clk='1') then
51             r_reg <= r_next;
52         end if;
53     end process;
54     -- next-state logic
55     r_next <= (others=>'0') when r_reg=9 else
56         r_reg + 1;
57     -- output logic
58     q <= std_logic_vector(r_reg);
59 end two_seg_arch;
```

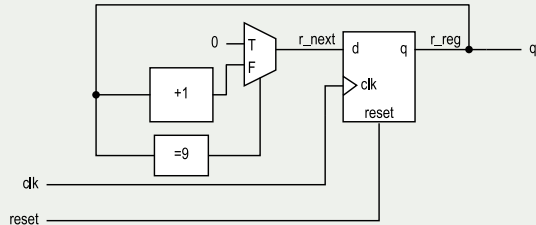


Uso indevido do *reset* assíncrono

■ Antes:



■ Depois:



Gated clock

- **Má prática:** usar uma porta lógica para desabilitar o *clock*, “parando” o registrador.
- **Exemplo:** Contador com um sinal de *enable*.



- **Má prática:** usar uma porta lógica para desabilitar o *clock*, “parando” o registrador.
- **Exemplo:** Contador com um sinal de *enable*.

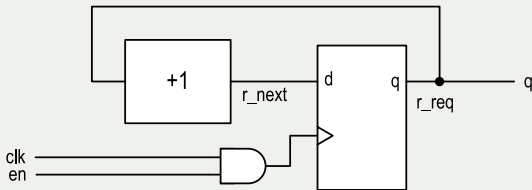
```
1  -----
2  -- Listing 9.3 gated clock
3  -----
4  library ieee;
5  use ieee.std_logic_1164.all;
6  use ieee.numeric_std.all;
7  entity binary_counter is
8      port(
9          clk, reset: in std_logic;
10         en: in std_logic;
11         q: out std_logic_vector(3 downto 0)
12     );
13 end binary_counter;
```

```
15 architecture gated_clk_arch of binary_counter is
16     signal r_reg: unsigned(3 downto 0);
17     signal r_next: unsigned(3 downto 0);
18     signal gated_clk: std_logic;
19 begin
20     -- register
21     process(gated_clk,reset)
22     begin
23         if (reset='1') then
24             r_reg <= (others=>'0');
25         elsif (gated_clk'event and gated_clk='1') then
26             r_reg <= r_next;
27         end if;
28     end process;
29     -- gated clock
30     gated_clk <= clk and en;
31     -- next-state logic
32     r_next <= r_reg + 1;
33     -- output logic
34     q <= std_logic_vector(r_reg);
35 end gated_clk_arch;
```



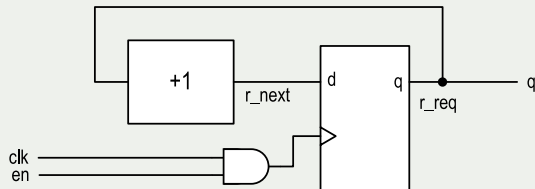
Gated clock

```
1  =====
2  -- Listing 9.3 gated clock
3  =====
4  library ieee;
5  use ieee.std_logic_1164.all;
6  use ieee.numeric_std.all;
7  entity binary_counter is
8      port(
9          clk, reset: in std_logic;
10         en: in std_logic;
11         q: out std_logic_vector(3 downto 0)
12     );
13 end binary_counter;
```



```
15 architecture gated_clk_arch of binary_counter is
16     signal r_reg: unsigned(3 downto 0);
17     signal r_next: unsigned(3 downto 0);
18     signal gated_clk: std_logic;
19 begin
20     -- register
21     process(gated_clk, reset)
22     begin
23         if (reset='1') then
24             r_reg <= (others=>'0');
25         elsif (gated_clk'event and gated_clk='1') then
26             r_reg <= r_next;
27         end if;
28     end process;
29     -- gated clock
30     gated_clk <= clk and en;
31     -- next-state logic
32     r_next <= r_reg + 1;
33     -- output logic
34     q <= std_logic_vector(r_reg);
35 end gated_clk_arch;
```





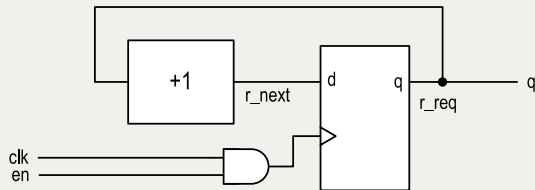
■ Problemas:

- A largura do *clock* resultante pode ser menor.
- O *clock* resultante pode sofrer com *glitches* do sinal de *enable*.
- Difícil de projetar a rede de distribuição do *clock*.

■ Regra:

- O *clock* **NÃO** deve ser manipulado.





■ Problemas:

- A largura do *clock* resultante pode ser menor.
- O *clock* resultante pode sofrer com *glitches* do sinal de *enable*.
- Difícil de projetar a rede de distribuição do *clock*.

■ Regra:

- O *clock* **NÃO** deve ser manipulado.

Solução:

Usar o *enable* síncrono.



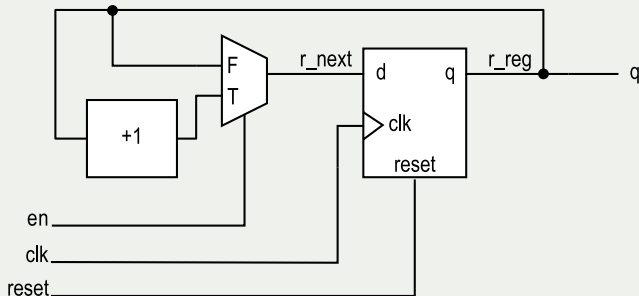
■ Solução:

```
37 -----
38 -- Listing 9.4
39 -----
40 architecture two_seg_arch of binary_counter is
41     signal r_reg: unsigned(3 downto 0);
42     signal r_next: unsigned(3 downto 0);
43 begin
44     -- register
45     process(clk,reset)
46     begin
47         if (reset='1') then
48             r_reg <= (others=>'0');
49         elsif (clk'event and clk='1') then
50             r_reg <= r_next;
51         end if;
52     end process;
53     -- next-state logic
54     r_next <= r_reg + 1 when en='1' else
55         r_reg;
56     -- output logic
57     q <= std_logic_vector(r_reg);
58 end two_seg_arch;
```

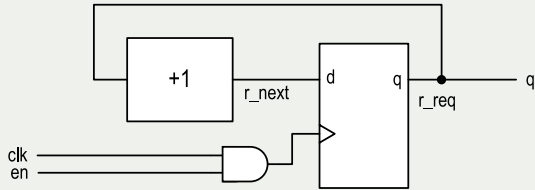


■ Solução:

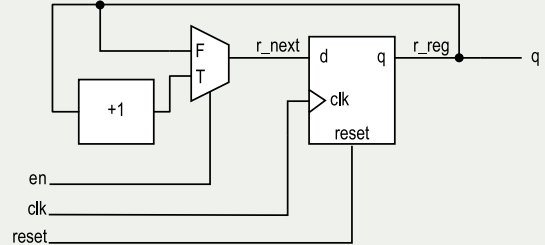
```
37 -----
38 -- Listing 9.4
39 -----
40 architecture two_seg_arch of binary_counter is
41     signal r_reg: unsigned(3 downto 0);
42     signal r_next: unsigned(3 downto 0);
43 begin
44     -- register
45     process(clk,reset)
46     begin
47         if (reset='1') then
48             r_reg <= (others=>'0');
49         elsif (clk'event and clk='1') then
50             r_reg <= r_next;
51         end if;
52     end process;
53     -- next-state logic
54     r_next <= r_reg + 1 when en='1' else
55         r_reg;
56     -- output logic
57     q <= std_logic_vector(r_reg);
58 end two_seg_arch;
```



■ Antes:

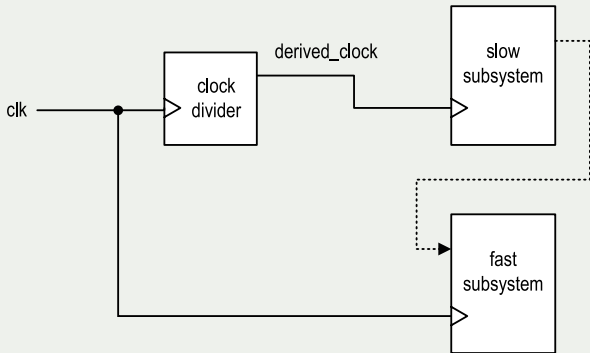


■ Depois:



Clock derivado

- Os subsistemas podem funcionar com *clocks* diferentes.
- **Má prática:** gerar um sinal mais lento para utilizar como *clock* de um subsistema.



- Problemas:
 - Gerenciar múltiplas redes de distribuição de *clock*.
 - “Deformação” do sinal de *clock*.
 - Como fazer a análise de *timing*? (taxa máxima de *clock*).
- O *clock* **NÃO** deve ser manipulado.



■ Problemas:

- Gerenciar múltiplas redes de distribuição de *clock*.
 - “Deformação” do sinal de *clock*.
 - Como fazer a análise de *timing*? (taxa máxima de *clock*).
- O *clock* **NÃO** deve ser manipulado.

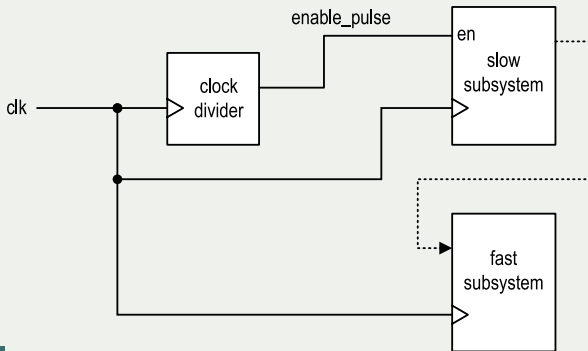
Solução:

Melhor usar pulso de *enable* síncrono.



■ Problemas:

- Gerenciar múltiplas redes de distribuição de *clock*.
 - “Deformação” do sinal de *clock*.
 - Como fazer a análise de *timing*? (taxa máxima de *clock*).
- O *clock* **NÃO** deve ser manipulado.



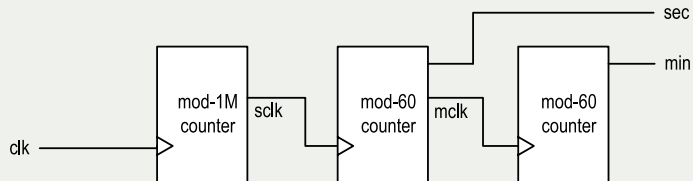
Solução:

Melhor usar pulso de *enable* síncrono.



Clock derivado

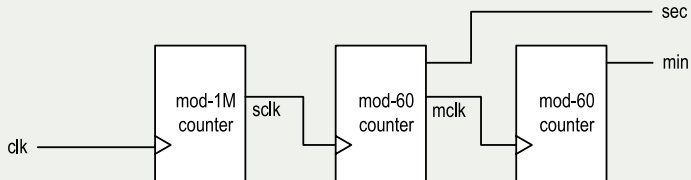
- Exemplo: Contador de minuto e segundo.
- *Clock* derivado:



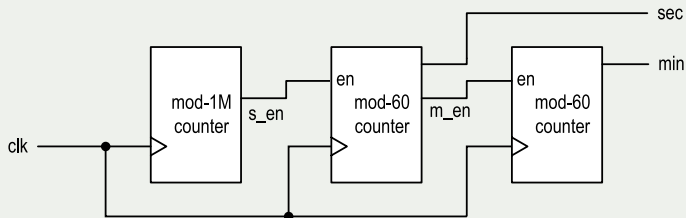
Clock derivado

- Exemplo: Contador de minuto e segundo.

- *Clock derivado:*



- Solução: *Clock síncrono.*



- Abrir código `list_09_05_06_timer.vhd`



Consumo de energia

- Atualmente, consumo de energia (*power*) é um dos principais critérios de projeto.
- Na tecnologia *CMOS*:
 - A potência dinâmica é proporcional à frequência de chaveamento dos transistores.
 - Taxa alta de *clock* implica em alta frequência de comutação.
- Dessa forma, manipulação de *clock*:
 - Pode reduzir a frequência de comutação
 - Mas não deve ser feito no nível RT



- Atualmente, consumo de energia (*power*) é um dos principais critérios de projeto.
- Na tecnologia *CMOS*:
 - A potência dinâmica é proporcional à frequência de chaveamento dos transistores.
 - Taxa alta de *clock* implica em alta frequência de comutação.
- Dessa forma, manipulação de *clock*:
 - Pode reduzir a frequência de comutação
 - Mas não deve ser feito no nível RT
- Fluxo de projeto:
 - 1 Projetar/sintetizar/verificar subsistemas síncronos regulares.
 - 2 *Clock* derivado: use circuito especial (PLL etc.) para obter *clocks* derivados.
 - 3 *Gated clock*: use a ferramenta de "otimização de energia" para converter alguns registros em *gated clock*.



Conclusão

- O reset assíncrono deve ser usado apenas para inicialização do sistema. Não deve ser usado para limpar os registros durante a operação normal.
- Não manipule ou bloqueie o sinal do clock. A operação desejada pode ser alcançada usando um registrador com um sinal de *enable*.



Fim!